Runnables

In the early phases, LangChain relied heavily on specific, rigid classes for every usecase (e.g. LLM chain, RetrievalQA Chain). This led to a code bloat and a steep learning curve because every chain had different methods (some used '.predict()', others '.run()', others '.format()').

Solution ⇒ The team rewrote the library using a standard interface called the **Runnable Protocol**. Now, every component (Prompt, Model, Retriever, Parser) inherits from the same base Runnable class.

* Every Runnable component must implement these core methods:

(a) .invoke() ⇒ Standard synchronous call for a single input.

(b) .batch() ⇒ Process a list of inputs in parallel.

(c) .stream() ⇒ Stream the output chunks.

(d) .ainvoke() ⇒ Async version of invoke.

(e) .abatch() ⇒ Async version of batch.

(f) .astream() ⇒ Async version of stream.

In web development (FastAPI / Django), blocking the code while waiting for an LLM (which is slow) is a bad practice.

Runnable Primitives & LCEL

There are two types of Runnables
- → Task-specific runnables (Models / Prompts)
- → Runnable Primitives (Utilities that help connect / route data)

1. Runnable Sequence (|) ⇒
It connects steps linearly (Output of Step A → Input of Step B).
LCEL syntax ⇒ Instead of manually passing variables, we use the pipe operator. $\boxed{\text{Chain} = \text{Prompt} \mid \text{Model} \mid \text{Parser}}$

2. Runnable Parallel ⇒ It runs multiple branches on the same input simultaneously. It returns a dictionary combining results from all branches. Eg. Generating a tweet and a LinkedIn post from the same topic at the same time.

Scanned with OKEN Scanner

3. Runnable Passthrough ⇒ It takes the input it receives and passes it to the output unchanged. Use case: Debugging (to see data mid-chain) or when using RunnableParallel where one branch needs to process data (e.g. calculate word count) while the other branch needs the original raw data.

4. Runnable Lambda ⇒ It converts any standard Python function into a Runnable. Use Case: Add custom logic (like text cleaning, sanitization, or word counting) into the middle of a chain.

5. Runnable Branch ⇒ It adds conditional logic (if/else) to chains. It evaluates a condition, if true, runs a specific Runnable. If false, runs a default/fallback Runnable.

***Automatic Coersion in LCEL:** LCEL is smart enough to coerce standard types automatically. If we put a dictionary in a chain {"key": chain}, it automatically converts to RunnableParallel. We don't always need to import the class explicitly.
Similarly if we use | my_function in a chain, it often automatically converts to RunnableLambda without us needing to wrap it manually.

*** Input/Output schemas in Runnables:** Every Runnable dynamically generates Pydantic schemas.
→ **.input_schema()** ⇒ Returns a JSON schema of what the chain expects.
→ **.output_schema()** ⇒ Returns a JSON schema of what the chain produces.
This allows automatic validation and is how tools like LangServe can automatically generate API documentation (Swagger UI) for our chains.

Document Loaders

These help us to connect and load data from different sources (PDFs, Databases, Web) into a standardized format that LangChain can process. Regardless of the source, the loader returns a **Document object** containing two keys:

1. **page_content:** The actual text data.
2. **metadata:** Info about the data (Source URL, page number, filepath)

Scanned with OKEN Scanner

There are numerous types of ~~Text~~ Document Loaders available in LangChain. Some of the most popular/used loaders are listed below.

(a) TextLoader ⇒ Simple loader for '.txt' files.

(b) ~~PyPDF~~ Directory Loader ⇒ Loads all files in a specific folder (and subfolders). We can also filter by filetype (eg., *.pdf).

(c) PyPDFLoader ⇒ Loads PDFs. Creates one Document object per page.

(d) WebBaseLoader ⇒ Uses 'BeautifulSoup' to scrape text from static websites.

(e) CSVLoader ⇒ Loads CSV files. Creates one Document object per row.

Eager Vs. Lazy Loading

* **.load()** (Eager) ⇒ Loads all documents into memory at once as a List. Good for small datasets.
* **.lazyload()** (Lazy) ⇒ Returns a Python generator. Loads documents one at a time on demand. Critical for massive datasets (eg. 1000s of PDFs) to prevent crashing RAM.

Note: LangChain also has integration with various libraries like 'Unstructured' which can help auto-detect file types and extract elements more robustly than standard loaders.

Text Splitters

Text splitters are used for breaking large documents into smaller chunks.

⇒ Why? LLM models have a maximum context window (token limit). Smaller chunks capture semantic meaning better than massive block of texts. There are various types of Text Splitters present in LangChain

(a) Character Text Splitter: This splits text strictly based on character counts. It is faster but bad because it can cut sentences or word in half, losing context.

(b) Recursive Character Text Splitter: It tries to keep related text together. It splits hierarchically in this order: Paragraphs (\n\n) → Sentences (\n) → Words ( ) → Characters.

Scanned with OKEN Scanner

(C) Code Splitters: These are specialized splitters which are based on specific coding languages like HTML, Python, or Markdown that respect syntax (eg. keeping a Python class definition together).

(d) Semantic Chunker: It splits text based on semantic meaning rather than the structure. It calculates cosine similarity between sentences and breaks the chunk when the topic changes (eg. switching from "Cricket" to "Farming").

Key Parameters for Splitters

⇒ Chunk Overlap: It is the buffer of text shared between two sequential chunks (usually 10% – 20%). This ensures context isn't lost if a sentence is cut right at the border of a chunk.

(e) Token Text Splitter: All the LLMs operate on tokens. A chunk of 1000 characters might be 200 tokens or 500 tokens depending on the language. For precise context management (like GPT-4's 8k limit), we can use Token Text Splitter to ensure our chunks fit exactly into the model's context window.

Vector Stores & Vector Databases

These are used for storing and searching through vector embeddings (numerical representations of text). Standard databases perform "Keyword Matching". Vector stores perform "Semantic Matching" (matching the meaning of the query to the data).

* Core Features: (a) Storage: Can be In-Memory (RAM, volatile) or on-Disk (Persistent). (b) Similarity Search: Uses cosine similarity to find vectors closest to the query vector. (c) Indexing: Uses algorithms (like HNSW) to search millions of vectors quickly without comparing them one by one.

⇒ Difference between Vector Store vs. Vector Database:
Vector stores are good for storage purposes and prototypes while Vector databases offer full-fledged system with CRUD support, backups, scaling and security features. All Vector Databases are Vector Stores but reverse is not true.

Scanned with OKEN Scanner